Conditionals and For Loops

The purpose of this notebook is to review (if you've seen this before) or learn about *if* statements and *for* loops in Python. Please read through the following material and run the code cells. There are opportunities to test your understanding throughout. Feel free to add cells and experiment along the way.

In [1]:
```
#Libraries
import numpy as np
import pandas as pd
```

Conditional Statements

There are some situations where we want to proceed with a task or perform an action dependent on whether a certain condition, or perhaps conditions, have been satisfied.

Conditional statements have the form of an "if-then" statement: *if* statement `P`, the hypothesis, occurs, *then* statement `Q`, the conclusion, also occurs.

How do we write this in code? We utilize the `if` expression in Python.

The statement below will execute the indented block *conclusion*, if the *hypothesis* is true; otherwise, if *hypothesis* is not true, then the indented block is ignored:

```
if hypothesis:
    conclusion
```

We implement an example by checking if a given number is positive. The below code says, `if` the provided x is greater than 0, the indented body will run, and 'Positive' is printed. Otherwise (if x < 0) the indented body is not executed.

```
if x > 0:
    print('Positive')
```

We can define a value for x and see what happens.

In [2]:
```
x = 5

if x > 0:
    print('Positive')
```

```
Positive
```

In [3]:
```
x = -6
```

```
if x > 0:
    print('Positive')
```

If we want to output something when x is a negative number we can do the following:

In [4]:
```
#we can keep our x value defined to be -6
x = -6

if x > 0:
    print('Positive')

if x < 0:
    print( 'Negative')
```

```
Negative
```

In the above code block, the first `if` condition was tested, it evaulated to False, and then the next `if` condition was tested, which evaulated to True, hence the body is executed - output is printed.

We can use `elif` statement instead.

Used in combination with an `if` expression, an `elif` statement is only checked if all previous statements evaluate to False. This allows us to check if our first condition is true, otherwise we move to evaluate the truth value of the next statement.

In [5]:
```
if x > 0:
    print('Positive')

elif x < 0:
    print('Negative')
```

```
Negative
```

We haven't addressed all possibilities! What happens when we define `x = 0`? Let's redefine the options to have an input in all cases.

In [6]:
```
#Now we can define x = 0
x=0

if x > 0:
    print('Positive')

elif x < 0:
    print('Negative')

elif x == 0:
    print('Neither positive nor negative')
```

```
Neither positive nor negative
```

Else statement

In an `elif` statement each condition is checked until a condition is true. Often we can replace the last `elif` statement with an `else` statement, whose body will be executed only if all the previous comparisons are false.

At this point, all other conditions have been evaluated, but none were executed. Thus, there is no condition or hypothesis associated with the `else` statement. If it is reached, the conclusion of the `else` statement is executed.

In [7]:
```
if x > 0:
    print('Positive')

elif x < 0:
    print('Negative')

else:
    print('Neither positive nor negative')
```

```
Neither positive nor negative
```

The General Format

```
if hypothesis_1:
    conclusion_1
else:
    conclusion_2
```

Or we could have $n + 1$ conclusions for a chosen n in which case we have the format:

```
if hypothesis_1:
    conclusion_1
elif hypothesis_2:
    conclusion_2
...
elif hypothesis_n:
    conclusion_n
else:
    conclusion
```

BE CAREFUL!!! Since the `else` statement is executed without checking a condition, you want to be absolutely certain that all desired possibilies are accounted for in the previous condition(s).

Exercise 1.

Use `if` statements to print your letter grade given a numerical grade where the following is true:

(a) Above a 90 is an A

(b) Between 80 and 90 is a B, including 80

(c) Between 70 and 80 is a C, including 70

(d) Between 60 and 70 is a D, including 60

(e) Below a 60 is an F

Check your code on the following grades:

my_grade = 83

my_grade = 70

my_grade = 57

In [ ]:
```
my_grade = 83

if my_grade >= 90:
    print("A")
elif my_grade >= 80:
    print("B")
elif my_grade >= 70:
    print("C")
elif my_grade >= 60:
    print("D")
else:
    print("F")
```

```
B
```

In [2]:
```
my_grade = 70

if my_grade >= 90:
    print("A")
elif my_grade >= 80:
    print("B")
elif my_grade >= 70:
    print("C")
elif my_grade >= 60:
    print("D")
else:
    print("F")
```

```
C
```

In [3]:
```
my_grade = 56

if my_grade >= 90:
    print("A")
elif my_grade >= 80:
    print("B")
elif my_grade >= 70:
    print("C")
elif my_grade >= 60:
    print("D")
else:
    print("F")
```

```
F
```

Exercise 2

Describe the difference in behavior between the following pieces of code. HINT: Try giving different values of x and executing the cell.

In [9]:
```
# CODE SEGMENT A
x = 10
if x > 0:
    print("x is positive")
elif x < -5:
    print("x is less than -5")
else:
    print("x is between -5 and 0")
```

```
x is positive
```

In [10]:
```
# CODE SEGMENT B
x = 10
if x > 0:
    print("x is positive")
if x < -5:
    print("x is less than -5")
else:
    print("x is between -5 and 0")
```

```
x is positive
x is between -5 and 0
```

Segment A uses elif and else, so the conditions are linked and only one of the three lines can print. Segment B has two separate if statements. The first if stands on its own, and the else belongs only to the second if. So for a positive x, A prints just "x is positive", while B prints "x is positive" and also "x is between -5 and 0", because the second if/else still runs on its own.

For Statements

We might be interested in repeating something 100 or 1000 times, or performing some action for each element in a list or array. If this is the case, we want to use `for` statements.

A `for` statement can *iterate* through a sequence to perform some action on each of its elements. The general form of a `for` statement is below:

```
for item in sequence:
    action
```

And for each of the *items* in *sequence* the indented body of the `for` statement is executed (here "action").

– or the indented body is "looped" –

This sequence can really be any *iterable* object, including a list, a string, or a range of numbers, to name a few.

Notice we specify a name to assign to the value of each of the sequence's items – here `item`. This name is assigned to each of the values `in` the sequence, sequentially.

An example

In [11]:
```
list_of_things= ["red", 2, 7.3, "dog"]
for element in list_of_things:
  print(element)
```

```
red
2
7.3
dog
```

This is equivalent to the following code.

In [12]:
```
element = "red"
print(element)

element = 2
print(element)

element = 7.3
print(element)

element = "dog"
print(element)
```

```
red
2
7.3
dog
```

In fact for relatively simple for loops,

```
for item in sequence:
    action
```

is equivalent to

```
item = sequence[0]
action

item = sequence[1]
action

item = sequence[2]
action
```

etc.

What can we iterate over?

Note that what we iterate over does not need to be directly related to the body of the `for` statement. In fact, `for` statements are useful to simply execute the body or action a given number of times.

In the first example below, the value of the *iterator*, i, is used in the body of the `for` statement; in the second example, the `for` statement uses this iterator merely to repeat or loop through the body statement a certain number of times.

In [13]:
```
for i in [1,2,3]:
    print(i)
```

```
1
2
3
```

We could also specify a *range* of values

In [14]:
```
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

In [15]:
```
for i in range(2,5): #we could choose the starting and ending values here too.
    print(i)
```

```
2
3
4
```

The iterator, i, does not have to appear in the body of the for statement, or for loop.

In [16]:
```
for i in range(3):
    print('hello')
```

```
hello
hello
hello
```

Often for loops are useful just to repeat something a specified number of times.

Exercise 3

Convert the following piece of a code to a for loop that does the same thing.

In [17]:
```
x = "I"
if len(x) > 3:
    print(x)

x = "only"
if len(x) > 3:
    print(x)

x = "use"
if len(x) > 3:
    print(x)

x = "long"
if len(x) > 3:
    print(x)

x = "words"
if len(x) > 3:
    print(x)
```

```
only
long
words
```

In [18]:
```
# Write your code in this cell.
l = ["I","only","use","long","words"]
for x in l:
    if len(x) > 3:
        print(x)
```

```
only
long
words
```

A Common Pattern

A common type of task you may find yourself doing is using a for loop to build a list.

```
result = []
for i in list:
```

```
  ...
  result.append(x)
```

The code below creates a list storing the first 100 perfect squares. (What is a perfect square again?....feel free to Google this if you don't know)

In [19]:
```
squares = []
for i in np.arange(0,101):
  squares.append(i**2)
squares[0:10]
```

Out[19]:
```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Similarly, the code below builds a list of the first 1000 Fibonacci numbers.

What is a Fibonacci number??!! They come from a sequence that follows the rule that each number in the sequence is the sum of the previous 2 numbers. Providing the first 2 numbers in the sequence (0, 1), you can find the next term in the sequence (0+1=1) to be 1, and the following term would be 2 (1+1 = 2). It might be tedious to find the 10th or 100th Fibonacci number, so we can use programming to help us!

In [20]:
```
fibonacci = [0,1]
for i in np.arange(2,1001):
  fibonacci.append(fibonacci[i-1]+fibonacci[i-2])
```

Warning

You may find yourself tempted to always use `np.array` for every list of numbers. However, they do have a disadvantage in this situation. It is significantly slower to append an element to a `np.array` than a list.

In [21]:
```
from time import process_time
t0 = process_time()
test_list = []
for i in np.arange(0,100000):
  test_list.append(i)

print("It took ", process_time()-t0, " seconds.")
```

```
It took  0.009714999999999918  seconds.
```

In [22]:
```
t0 = process_time()
test_list = np.array([])
for i in np.arange(0,100000):
  test_list = np.append(test_list,i)

print("It took ", process_time()-t0, " seconds.")
```

```
It took  1.2430610000000004  seconds.
```

Nested *for* loops

Suppose we have two lists, and we want to pair every element in `list_1` with every element in `list_2`.

This takes the following form:

```
for item_1 in list_1:
    for item_2 in list_2:
        print(item_1, item_2)
```

We *could* write out by hand all the possible combinations pairing elements of `list_1` with `list_2` ... or, we could use nested `for` statements to systematically consider each element in `list_2` for each element in `list_1`.

Example

Suppose we want to find all possible combinations of the below lists.

In [23]:
```
my_animals = ['dog', 'cat', 'cow']

adjectives = ['hairy', 'scary', 'cute']

for adj in adjectives:
    for animal in my_animals:
        print(adj, animal)
```

```
hairy dog
hairy cat
hairy cow
scary dog
scary cat
scary cow
cute dog
cute cat
cute cow
```

Order of the inner and outer *for* loops does matter!

In [24]:
```
for animal in my_animals:
    for adj in adjectives:
        print(adj, animal)
```

```
hairy dog
scary dog
cute dog
hairy cat
scary cat
cute cat
hairy cow
scary cow
cute cow
```

Exercise 4

Write code to print the first 10 natural numbers (positive whole numbers starting at 1)

Describe your thought process

In [25]:
```
# Loop over the numbers 1 through 10 and print each one.
for i in range(1, 11):
    print(i)
```

```
1
2
3
4
5
6
7
8
9
10
```

Exercise 5

Use a for loop and the function `np.sqrt` to build a list containing the square roots of the first 10,000 natural numbers.

In [26]:
```
roots = []
for i in np.arange(1, 10001):
    roots.append(np.sqrt(i))
roots[:10]
```

Out[26]:
```
[1.0,
 1.4142135623730951,
 1.7320508075688772,
 2.0,
 2.23606797749979,
 2.449489742783178,
 2.6457513110645907,
 2.8284271247461903,
 3.0,
 3.1622776601683795]
```

Exercise 6

Write code to find the sum of the numbers 1-100 (You should get 5050).

Describe your thought process

In [ ]:
```
# I add each number from 1 to 100 to a running total.
total = 0
for i in range(1, 101):
    total = total + i
print(total)
```

```
5050
```

Exercise 7

Write code to display the pattern below:

```
*
**
***
****
```

Describe your thought process

In [ ]:
```
# I print i stars on each row, for i from 1 to 4.
for i in range(1, 5):
    print("*" * i)
```

```
*
**
***
****
```

Exercise 8

Write code to display the pattern below:

```
1
12
123
1234
```

Describe your thought process

In [ ]:
```
#  I have to build each row by adding the next number as text, then print
it.
row = ""
for i in range(1, 5):
    row = row + str(i)
    print(row)
```

```
1
12
123
1234
```

Exercise 9

Write code that will take a string, split it up into individual words, and count the number of capitalized words. It should work for any sentence. HINT: Search the internet to find a string method that can determine if a world is capitalized or not.

In [30]:
```
s = "This sentence written by Bill Trok in Chicago, Illinois has five
capitalized words."
words = s.split()
# Your Code Here
count = 0
for w in words:
    if w.istitle():
        count = count + 1
print(count)
```

```
5
```